تمرین کلاس — بالا آوردن سرویس FastAPI با Docker

امروز یک سرویس کوچک می‌سازیم و همان را داخل Docker اجرا می‌کنیم. مدل این تمرین شبکه عصبی نیست. یک تابع ساده است که با چند کلمه، متن را مثبت یا منفی می‌کند. برای همین GPU لازم نیست و کار روی CPU لپ‌تاپ انجام می‌شود.

پوشه کار را خودتان بسازید. اسم پوشه: `classroom-api`

قبل از شروع، این دو دستور را بزنید و خروجی را ببینید:

```
python3 --version
docker version
```

Python باید ۳.۱۱ یا جدیدتر باشد. اگر دستور دوم خطا داد، Docker Desktop را باز کنید و صبر کنید تا کاملاً بالا بیاید. نصب بودن Docker به‌تنهایی کافی نیست.

چند تعریف

`endpoint` آدرسی روی سرویس است که یک کار مشخص انجام می‌دهد. امروز `/healthz` و `/readyz` و `/predict` را می‌سازیم.

`contract` شکل دقیق درخواست و پاسخ است. آن را با Pydantic می‌نویسیم. اگر ورودی با این شکل نخواند، FastAPI خودش کد ۴۲۲ برمی‌گرداند.

`model adapter` تابع `run_local_text_model` است. مسیر `/predict` فقط همین تابع را صدا می‌زند. اگر بعداً به‌جای این تابع مدل دیگری بگذارید، شکل درخواست و پاسخ عوض نمی‌شود.

`/healthz` می‌پرسد خود برنامه زنده است و جواب می‌دهد یا نه. این مسیر مدل را اجرا نمی‌کند.

`/readyz` می‌پرسد مدل آماده است و می‌شود از آن پیش‌بینی خواست یا نه. اگر مدل آماده نباشد، کد ۵۰۳ برمی‌گردد.

`status code` عدد جواب HTTP است.



کد ۲۰۰ یعنی OK. درخواست به سرور رسیده و جواب موفق برگشته است.

کد ۴۲۲ یعنی ورودی با قراردادی که نوشتید نمی‌خواند. کد ۵۰۳ یعنی برنامه زنده است، ولی مدل فعلاً آماده نیست. کد ۵۰۰ یعنی خطای داخلی. متن خطای داخلی مدل را در جواب به کاربر ننویسید.

`image` خروجی دستور `docker build` است. بسته آماده برنامه و کتابخانه‌هاست و هنوز اجرا نشده است.

`container` همان image است وقتی با `docker run` اجرایش می‌کنید. یک پروسه در حال اجراست.

`0.0.0.0` یعنی Uvicorn روی همه آدرس‌های همان ماشین گوش بدهد. اگر داخل container فقط روی `127.0.0.1` گوش بدهد، درخواست از لپ‌تاپ به آن نمی‌رسد.

فایل‌هایی که در پوشه می‌سازید:

```
classroom-api/
  main.py
  test_service.py
  requirements.txt
  Dockerfile
  .dockerignore
```

۱. محیط پایتون را روی لپ‌تاپ بسازید

داخل پوشه `classroom-api` این دستورها را بزنید:

```
python3 -m venv .venv
source .venv/bin/activate
```

در ویندوز به‌جای خط دوم:

```
.venv\Scripts\activate
```

فایل `requirements.txt` را با همین سه خط بسازید:

```
fastapi==0.116.1
uvicorn==0.35.0
httpx==0.28.1
```

علامت `==` نسخه را ثابت می‌کند تا روی لپ‌تاپ همه همان نسخه نصب شود. اگر نسخه را ننویسید، ممکن است روز دیگری بسته دیگری نصب شود.

```
pip install -r requirements.txt
```

۲. سرویس را در فایل main.py بنویسید

این فایل را کامل در `main.py` بگذارید. پایین‌تر یک فیلد به کلاس درخواست اضافه می‌کنید.

```
import os
import time
import uuid
from typing import Any

from fastapi import FastAPI, HTTPException, Request
from pydantic import BaseModel, Field

APP_VERSION = "0.1.0"
MODEL_STATUS = "ready"
MAX_TEXT_CHARS = 512
SERVICE_NAME = "classroom-model-api"

app = FastAPI(title="Classroom Model API", version=APP_VERSION)

POSITIVE_WORDS = {"love", "great", "excellent", "fast", "helpful", "good"}
NEGATIVE_WORDS = {"hate", "bad", "slow", "broken", "wrong", "terrible"}


class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=MAX_TEXT_CHARS)
    request_id: str | None = Field(default=None, max_length=64)
```

```
class PredictResponse(BaseModel):
    request_id: str
    label: str
    score: float = Field(..., ge=0.0, le=1.0)
    model: str
    latency_ms: float = Field(..., ge=0.0)


def run_local_text_model(text: str) -> dict[str, Any]:
    normalized = text.lower()
    if "[model_down]" in normalized:
        raise RuntimeError("simulated local model outage")

    tokens = set(normalized.replace(".", " ").replace(",", " ").split())
    positive_hits = len(tokens & POSITIVE_WORDS)
    negative_hits = len(tokens & NEGATIVE_WORDS)
    raw_score = 0.5 + 0.15 * (positive_hits - negative_hits)
    score = min(0.95, max(0.05, raw_score))
    label = "positive" if score >= 0.5 else "negative"
    return {
        "label": label,
        "score": round(score, 3),
        "model": "demo-sentiment-v1",
    }


@app.middleware("http")
async def add_version_header(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Service-Version"] = APP_VERSION
    return response


@app.get("/healthz")
async def healthz():
    return {
        "status": "ok",
        "service": SERVICE_NAME,
        "version": APP_VERSION,
        "environment": os.getenv("APP_ENV", "local"),
    }


@app.get("/readyz")
async def readyz():
    if MODEL_STATUS != "ready":
        raise HTTPException(
            status_code=503,
            detail="model dependencies are not ready",
        )
    return {
        "status": "ready",
        "model_status": MODEL_STATUS,
        "version": APP_VERSION,
    }


@app.post("/predict", response_model=PredictResponse)
async def predict(request: PredictRequest):
    started = time.perf_counter()
    request_id = request.request_id or f"req-{uuid.uuid4().hex[:10]}"

    if not request.text.strip():
        raise HTTPException(
            status_code=422,
            detail="text must contain a non-whitespace character",
```

```
        )

    print(f"predict request_id={request_id} text_len={len(request.text)}")

    try:
        result = run_local_text_model(request.text)
    except RuntimeError:
        raise HTTPException(status_code=503, detail="model is temporarily unavailable")
    except Exception:
        raise HTTPException(status_code=500, detail="prediction failed")

    latency_ms = round((time.perf_counter() - started) * 1000, 3)
    return PredictResponse(
        request_id=request_id,
        latency_ms=latency_ms,
        **result,
    )
```

اگر متن شامل `[model_down]` باشد، تابع عمداً خطا می‌دهد تا کد ۵۰۳ را ببینید. خط `print` خود متن کاربر را در لاگ نمی‌نویسد و فقط شناسه درخواست و طول متن را می‌نویسد. به این کار redaction می‌گوییم، یعنی اطلاعات حساس را از لاگ حذف کردن.

این فیلد را داخل کلاس `PredictRequest`، بعد از `request_id`، اضافه کنید:

```
language: str | None = Field(default=None, max_length=16)
```

مدل از این فیلد استفاده نمی‌کند. اضافه شدنش یعنی شکل درخواست را خودتان کمی کامل‌تر کرده‌اید و کلاینت می‌تواند زبان متن را هم بفرستد.

۳. سرور را روی لپ‌تاپ بالا بیاورید

در همان پوشه و با همان محیط مجازی:

```
uvicorn main:app --reload --port 8000
```

`--reload` یعنی با هر بار ذخیره فایل، سرور دوباره بالا بیاید. این گزینه فقط برای کار روی لپ‌تاپ است. در Docker از آن استفاده نکنید. این ترمینال را نبندید.

در مرورگر این آدرس را باز کنید:

```
http://127.0.0.1:8000/docs
```

این صفحه همان شکل درخواست و پاسخ را نشان می‌دهد.

ترمینال دوم را باز کنید، دوباره بروید داخل `classroom-api`، و این درخواست‌ها را یکی‌یکی بزنید:

```
curl http://127.0.0.1:8000/healthz
curl http://127.0.0.1:8000/readyz
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"The assistant is fast and helpful.","language":"en"}'
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"The response is slow and broken."}'
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
```

```
  -d '{}'
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"please use [MODEL_DOWN] now"}'
```

این‌ها را در خروجی چک کنید:

- `/healthz` و `/readyz` کد ۲۰۰ می‌دهند.
- جمله `fast and helpful` برچسب `positive` می‌گیرد.
- جمله `slow and broken` برچسب `negative` می‌گیرد.
- درخواست بدون فیلد `text` کد ۴۲۲ می‌دهد.
- متنی که `[MODEL_DOWN]` دارد کد ۵۰۳ می‌دهد.
- در جواب ۵۰۳ نباید عبارت `simulated local model outage` باشد. آن عبارت خطای داخلی است و نباید به کاربر برسد.
- در هدر جواب، `X-Service-Version` برابر `0.1.0` است.

برای دیدن هدر:

```
curl -i http://127.0.0.1:8000/healthz
```



این آدرس را هم بزنید. چنین مسیری را در برنامه نساخته‌اید:

```
curl -i http://127.0.0.1:8000/no-such-path
```

باید کد ۴۰۴ بگیرید. معنی آن Not Found است: این آدرس روی سرور وجود ندارد. سرور خاموش نیست و مدل هم خراب نیست. فقط این مسیر تعریف نشده است.

حالا در `main.py` مقدار `MODEL_STATUS` را از `"ready"` به `"loading"` عوض کنید و فایل را ذخیره کنید. چون `--reload` روشن است، لازم نیست سرور را دستی قطع و وصل کنید. دوباره هر دو مسیر را بزنید.

`/healthz` هنوز ۲۰۰ است، چون خود برنامه زنده است. `/readyz` کد ۵۰۳ می‌دهد، چون مدل را آماده حساب نکرده‌اید.

بعد `MODEL_STATUS` را به `"ready"` برگردانید و صبر کنید `/readyz` دوباره ۲۰۰ شود. اگر این کار را نکنید، تست بخش ۴ خطا می‌گیرد.

۴. همان سرویس را بدون Uvicorn تست کنید

TestClient درخواست را از داخل خود پایتون می‌فرستد و لازم نیست سرور بخش ۳ روشن باشد. اگر Uvicorn هنوز روشن است، در ترمینال آن `Ctrl+C` بزنید.

فایل `test_service.py`:

```
from fastapi.testclient import TestClient
```

```
from main import app

client = TestClient(app)


def main():
    health = client.get("/healthz")
    ready = client.get("/readyz")
    ok = client.post(
        "/predict",
        json={"text": "This service is fast and helpful.", "language": "en"},
    )
    invalid = client.post("/predict", json={})
    down = client.post("/predict", json={"text": "please use [MODEL_DOWN] now"})

    assert health.status_code == 200
    assert ready.status_code == 200
    assert ok.status_code == 200
    assert ok.json()["label"] == "positive"
    assert "language" not in ok.json()
    assert invalid.status_code == 422
    assert down.status_code == 503
    assert "simulated local model outage" not in down.text
    assert health.headers["X-Service-Version"] == "0.1.0"
    print("all checks passed")


if __name__ == "__main__":
    main()
```

```
python test_service.py
```

باید این جمله چاپ شود: `all checks passed`

۵. همین سرویس را با Docker اجرا کنید

اگر Uvicorn لپ‌تاپ هنوز روشن است آن را خاموش کنید. برنامه روی لپ‌تاپ و container نمی‌توانند همزمان پورت ۸۰۰۰ لپ‌تاپ را بگیرند.

فایل `.dockerignore`:

```
.venv
__pycache__
.git
.env
```

این‌ها داخل image کپی نمی‌شوند. پوشه `.venv` مال لپ‌تاپ شماست. فایل `.env` جای رمز است و نباید داخل image برود.

فایل `Dockerfile`:

```
FROM python:3.12.3-slim-bookworm
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

خط `FROM` می‌گوید ساخت image از کدام پایتون شروع شود. شماره نسخه نوشته شده تا بعداً تصویر دیگری جایش را نگیرد.

نصب بسته‌ها قبل از `USER` انجام می‌شود، چون نصب به کاربر root نیاز دارد. خود برنامه با کاربر معمولی `appuser` اجرا می‌شود.

`EXPOSE` فقط اعلام می‌کند برنامه داخل container روی پورت ۸۰۰۰ کار می‌کند. باز شدن این پورت روی لپ‌تاپ با `-p` در دستور اجراست.

رمز را با `ENV` داخل Dockerfile ننویسید. هر چه آنجا باشد داخل image می‌ماند.

از داخل `classroom-api` :

```
docker build -t classroom-api:0.1.0 .
```

بار اول، Docker تصویر پایه پایتون را دانلود می‌کند و کمی طول می‌کشد. مدل و GPU دانلود نمی‌شود.

```
docker run --rm -p 8000:8000 classroom-api:0.1.0
```

در `-p 8000:8000` عدد سمت چپ پورت لپ‌تاپ است و عدد سمت راست پورت داخل container. `--rm` یعنی با خاموش شدن، container پاک شود. image پاک نمی‌شود.

ترمینال دیگری باز کنید و این سه درخواست را بزنید:

```
curl http://127.0.0.1:8000/healthz
curl http://127.0.0.1:8000/readyz
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"The assistant is fast and helpful."}'
```

در ترمینال `docker run` باید `text_len` را ببینید. خود جمله کاربر نباید در لاگ باشد. بعد این container را با `Ctrl+C` خاموش کنید.

همان image را با یک تنظیم دیگر اجرا کنید. دوباره `build` نکنید:

```
docker run --rm -p 8000:8000 -e APP_ENV=staging classroom-api:0.1.0
```

```
curl http://127.0.0.1:8000/healthz
```

در جواب، `environment` باید `staging` باشد. چیزی که از یک محیط به محیط دیگر فرق دارد با `-e` موقع اجرا می‌آید. برای عوض کردنش image تازه نمی‌سازیم. این container را هم با `Ctrl+C` خاموش کنید.

۶. ببینید چرا درخواست از لپ‌تاپ به container نمی‌رسد

در این بخش یک اشتباه رایج را خودتان می‌سازید تا دلیلش را ببینید.

در Dockerfile آرگومان `--host` را بردارید. خط `CMD` باید این‌طور شود:

```
CMD ["uvicorn", "main:app", "--port", "8000"]
```

image را دوباره بسازید و container را اجرا کنید:

```
docker build -t classroom-api:0.1.0 .
docker run --rm -p 8000:8000 classroom-api:0.1.0
```

```
curl http://127.0.0.1:8000/healthz
```

از لپ‌تاپ جواب نمی‌گیرید، ولی در ترمینال `docker run` برنامه روشن است.

دلیل: اگر `--host` را ننویسید، Uvicorn فقط روی `127.0.0.1` داخل همان container گوش می‌دهد. دستور `-p` درخواست لپ‌تاپ را به آدرس شبکه container می‌فرستد، نه به آن آدرس داخلی. برای همین از بیرون به سرویس نمی‌رسید.

`--host 0.0.0.0` را به خط `CMD` برگردانید، image را دوباره بسازید، container را اجرا کنید، و `curl` را تکرار کنید تا کد ۲۰۰ برگردد. این container را روشن نگه دارید. بخش ۷ به آن نیاز دارد.

۷. سه جمله را بفرستید و درستی جواب را حساب کنید

container باید روشن باشد و `--host 0.0.0.0` را داشته باشد. اگر آن را خاموش کرده‌اید، اول این را بزنید و صبر کنید سرویس بالا بیاید:

```
docker run --rm -p 8000:8000 classroom-api:0.1.0
```

در ترمینال دیگری سه درخواست زیر را جدا جدا بزنید. در JSON هر جواب، فیلد `label` را پیدا کنید و با انتظار همان جمله مقایسه کنید. اگر `curl` وصل نشد، اول مطمئن شوید container روشن است و پورت ۸۰۰۰ لپ‌تاپ را برنامه دیگری نگرفته است.

جمله اول. انتظار برچسب: `positive`

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"The assistant is fast and helpful."}'
```

جمله دوم. انتظار برچسب: `negative`

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"The response is slow and broken."}'
```

جمله سوم. انتظار برچسب: `neutral`

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"A neutral statement."}'
```

`accuracy` یعنی تعداد جمله‌هایی که `label` آن‌ها با انتظار یکی است، تقسیم بر ۳. جمله سوم هیچ کلمه‌ای از فهرست مثبت یا منفی ندارد. امتیاز روی ۰.۵ می‌ماند و تابع آن را `positive` برمی‌گرداند، در حالی که انتظار ما `neutral` بود. پس این یکی نادرست است.

سرور می‌تواند روشن باشد و `/healthz` کد ۲۰۰ بدهد، ولی برچسب اشتباه برگرداند. در درس به این می‌گوییم خطای quality، یعنی کیفیت جواب. این با خاموش بودن سرویس فرق دارد.

تأخیر را روی کاغذ حساب کنید

این ده عدد میلی‌ثانیه را به سرور نمی‌فرستید. فقط حساب می‌کنید:

```
20, 22, 25, 28, 30, 31, 40, 55, 80, 2000
```

اول میانگین را حساب کنید. فهرست از کوچک به بزرگ مرتب است. برای صدک، اندیس از صفر شروع می‌شود:

```
index = ceil(percentage / 100 * n) - 1
```

اینجا `n` تعداد عددهاست و برابر ۱۰ است.

برای p50 مقدار `percentage` را ۵۰ بگذارید. اندیس ۴ می‌شود، یعنی پنجمین عدد. برای p95 مقدار را ۹۵ بگذارید. اندیس ۹ می‌شود، یعنی دهمین عدد. هر دو را بنویسید و با میانگین مقایسه کنید. اگر فقط میانگین را ببینید، کندی آخرین درخواست دیده نمی‌شود.

حافظه وزن یک مدل واقعی را حساب کنید

این حساب جدا از سرویسی است که بالا آوردید. مدل امروز وزن و پارامتر ندارد.

بایت وزن ≈ تعداد پارامتر × تعداد بیت هر پارامتر / ۸
گیگابایت ≈ همان بایت / ۱۰۲۴ به توان ۳

برای مدلی با ۷ میلیارد پارامتر، حجم وزن ۱۶ بیتی و حجم وزن ۴ بیتی را جدا حساب کنید. بگویید کدام‌یک، فقط از نظر فایل وزن، در ۸ گیگابایت جا می‌شود. جا شدن وزن کافی نیست. حافظه میانی محاسبه مدل (activation) و سربار خود برنامه هم اضافه می‌شود. چون بیشتر لپ‌تاپ‌ها GPU ندارند، امروز مدل عصبی اجرا نکردیم.

۸. کدهای خطای تصویر را وصل کنید

در این تمرین مسیر آپلود تصویر نمی‌سازید. هر وضعیت را به کد روبه‌رویش وصل کنید و در پاسخ‌نامه بنویسید.

• فایل اصلاً تصویر نیست: ۴۱۵



حجم فایل، یا عرض و ارتفاع تصویر، از حد مجاز سرویس بیشتر است: ۴۱۳. معنی آن Payload Too Large است. بدنه درخواست برای این سرویس بیش از حد بزرگ است.

- نوع فایل تصویر است، ولی محتوایش تصویر قابل خواندن نیست: ۴۰۰
- مدل موقتاً در دسترس نیست: ۵۰۳

۹. پاسخ‌نامه را بنویسید

فایل `answers.txt` را در همان پوشه پر کنید.

1. `/healthz` و `/readyz` هر کدام چه سوالی را جواب می‌دهند؟ وقتی `MODEL_STATUS` برابر `loading` بود، هر کدام چه کدی دادند؟
2. مسیر `/no-such-path` چه کدی داد؟ این کد با ۵۰۳ چه فرقی دارد؟
3. `image` و `container` چه فرقی دارند؟ در `-p 8000:8000` کدام عدد مال لپ‌تاپ است؟
4. چرا بدون `--host 0.0.0.0` از لپ‌تاپ به container نمی‌رسیدید؟
5. چرا شماره نسخه پایتون و نسخه fastapi را در فایل‌ها ثابت کردید؟
6. میانگین، p50 و p95 همان ده عدد را بنویسید. چرا میانگین به‌تنهایی گمراه‌کننده است؟
7. حجم وزن ۱۶ بیتی و ۴ بیتی مدل ۷ میلیارد پارامتری را به گیگابایت بنویسید. کدام در ۸ گیگابایت جا می‌شود و چه حافظه دیگری هنوز مانده است؟
8. `accuracy` سه جمله بخش ۷ چند شد؟ اگر `/healthz` کد ۲۰۰ بدهد، این چه نوع خطایی است؟
9. چهار وضعیت بخش ۸ را با کدهایشان بنویسید.
10. اگر دیروز بیشتر متن‌ها انگلیسی بود و امروز بیشتر فارسی است، و `/healthz` هنوز ۲۰۰ است، کدام تصمیم درست است: `SHIP` یا `INVESTIGATE` یا `BLOCK` ؟ یک دلیل بنویسید.

تحویل، پوشه `classroom-api` است: کد، Dockerfile و `answers.txt` . خروجی `curl` بخش ۳ و بخش ۷ را هم در پاسخ‌نامه کپی کنید.

اگر گیر کردید

- `docker version` خطا می‌دهد: Docker Desktop بسته است یا هنوز بالا نیامده.
- پیام اشغال بودن پورت: Uvicorn لپ‌تاپ هنوز روشن است، یا container قبلی را خاموش نکرده‌اید.
- `curl` در بخش ۶ جواب نمی‌دهد: همان برداشتن `--host` است، به شرطی که `-p 8000:8000` را گذاشته باشید. این حالت ۴۰۴ نیست. در ۴۰۴ سرور جواب می‌دهد و می‌گوید مسیر را نمی‌شناسد.
- container بلافاصله خاموش می‌شود: در `CMD` باید `main:app` باشد، چون اسم فایل `main.py` است.
- داخل container نوشته `ModuleNotFoundError`: بسته‌ها باید از `requirements.txt` داخل image نصب شده باشند، نه از `.venv` لپ‌تاپ.
- `build` طولانی است: بار اول تصویر پایه پایتون دانلود می‌شود. مدل یا GPU دانلود نمی‌کنید.